



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	MJ Programming from First Principles	
Topic Name	Standard Data Types-Numbers (practice session) part 2	
Topic Code	-	
Unit/Module No. & Name	2	Standard Data Types
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	5
2.0	Meaning And Definition	6-8
3.0	Nature Or Type	9-10
4.0	Examples And Case Studies	11-14
5.0	Keywords And Touch Vocabulary	15



OBJECTIVE

1. This unit explains how computers work with numbers at the binary level.
2. It focuses on binary addition and binary subtraction.
3. It also explains how negative numbers are represented using two's complement.
4. The language is kept simple so that each step is easy to understand.
5. This helps even students who feel nervous about mathematics.
6. At the same time, the tone remains academic and suitable for undergraduate and online MCA study.
7. The first purpose of this unit is to show that binary arithmetic follows clear rules.
8. These rules are similar to the rules of decimal arithmetic.
9. When you understand the rules, you can solve problems with confidence.
10. The second purpose is to connect these rules to real computer behavior.
11. When a program adds two integers, the computer uses these binary rules.
12. The same ideas are used when a program subtracts values such as balances.
13. The third purpose is to prepare you for subjects like computer organization.
14. It also prepares you for digital logic and system design.
15. In these subjects, binary numbers play a central role.
16. After studying this unit, you should be able to add binary numbers confidently.
17. You should also be able to subtract binary numbers correctly.
18. You should be able to write the one's complement of a small binary number.
19. You should be able to write the two's complement of a small binary number.

20. You should know how two's complement represents negative integers using a fixed number of bits.
21. You should be able to explain what overflow means in two's complement arithmetic.
22. You should also be able to check your own calculations.
23. You should be able to find and correct simple mistakes.
24. For online MCA students, these skills are useful in many courses.
25. When you design an algorithm, you may need to consider the range of values in a data type.
26. When you write low-level code in C, these ideas become important.
27. They are also important when working with embedded systems.
28. You must understand what happens when a calculation goes beyond the allowed range.
29. This unit provides a strong foundation for handling such practical problems later.



1.0 INTRODUCTION

1.1 Numbers In Daily Life And In Computers

In daily life, we use decimal numbers without much thought. We use numbers to count money, time, marks, and distance using the digits zero to nine. Computers, however, do not use decimal digits directly. Inside a computer, all information is stored as bits. Each bit can be either zero or one. A group of bits can represent a number, a letter, a color, or even part of a picture or video.

Because computer hardware works with bits, computer arithmetic uses the binary number system. In binary, each position represents a power of two instead of a power of ten. The rightmost bit has a value of one, the next bit has a value of two, then four, eight, and so on. When you do binary arithmetic on paper, you are following the same steps that the processor performs electronically. Understanding this helps you think like a computer when writing or fixing programs.

1.2 From Decimal Thinking To Binary Thinking

Many students feel that binary arithmetic is confusing at first. They are comfortable adding and subtracting in base ten but not in base two. The good news is that the idea is almost the same. You already know how to add and subtract numbers; you just need to change the base. In decimal, ten ones make one ten. In binary, two ones make one two. Once you understand this rule, the rest becomes easier.

In practice, you will often convert decimal numbers into binary and then work with them. For example, decimal five becomes 0101 in four-bit binary, and decimal minus three becomes 1101 in four-bit two's complement form. When you add these bit patterns, you are working exactly as the computer stores and processes numbers. With practice, you start noticing patterns, such as negative numbers having a one in the leftmost bit when using two's complement.

1.3 Role Of Practice Sessions

Practice sessions, like the one in this unit, help you apply theory to many small problems. Solving simple additions and subtractions builds speed and confidence. When you solve many short binary problems, the rules start to feel natural. They move from short-term memory to long-term memory, making them easier to recall during exams or interviews.

Practice also helps you find your mistakes. For example, you might forget to carry a one or flip bits incorrectly while finding a complement. By checking correct solutions, you can see where you went wrong. In later problems, you become more careful with those steps. In this way, practice creates a feedback loop that improves both accuracy and confidence.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Binary Arithmetic

Binary arithmetic means doing calculations on numbers written in base two. The main operations are addition and subtraction, though multiplication and division are also possible. In binary addition, you only have four basic cases: zero plus zero, zero plus one, one plus zero, and one plus one. In binary subtraction, you borrow from the next bit when needed, just as you borrow from the next digit in decimal subtraction.

Because the rules are simple, binary arithmetic is well suited to hardware. Logic gates like AND, OR, and XOR can be wired together to build an adder circuit that always obeys these rules. When millions of add operations are required for a multimedia program or a machine learning model, hardware can perform them quickly and reliably. As a student, you learn the paper method so that you can understand what the circuits are doing.

2.2 Definition Of Signed Binary Numbers

So far we have talked mainly about positive numbers. In many real cases, we also need negative numbers. Temperatures can go below zero, bank balances can go negative, and vectors in graphics can point in negative directions. To handle this need, we define signed binary numbers. A signed binary number has a pattern of bits that encodes both the size and the sign of the value.

There are several systems for signed binary form. The main ones are sign magnitude, one's complement, and two's complement. In sign magnitude, the leftmost bit shows the sign: zero for positive and one for negative. The remaining bits give the magnitude, or absolute value. In one's complement and two's complement, we change the bit pattern more deeply to encode negatives in a way that works well with binary addition and subtraction.

2.3 Two's Complement Representation

Two's complement is the most common system for signed integers in modern computers. To find the two's complement of a binary number, you follow two steps. First you flip every bit, turning zero to one and one to zero. This step gives the one's complement. Then you add one to the result. The final pattern is the two's complement. This pattern represents the negative of the original number, assuming we stay within a fixed number of bits.

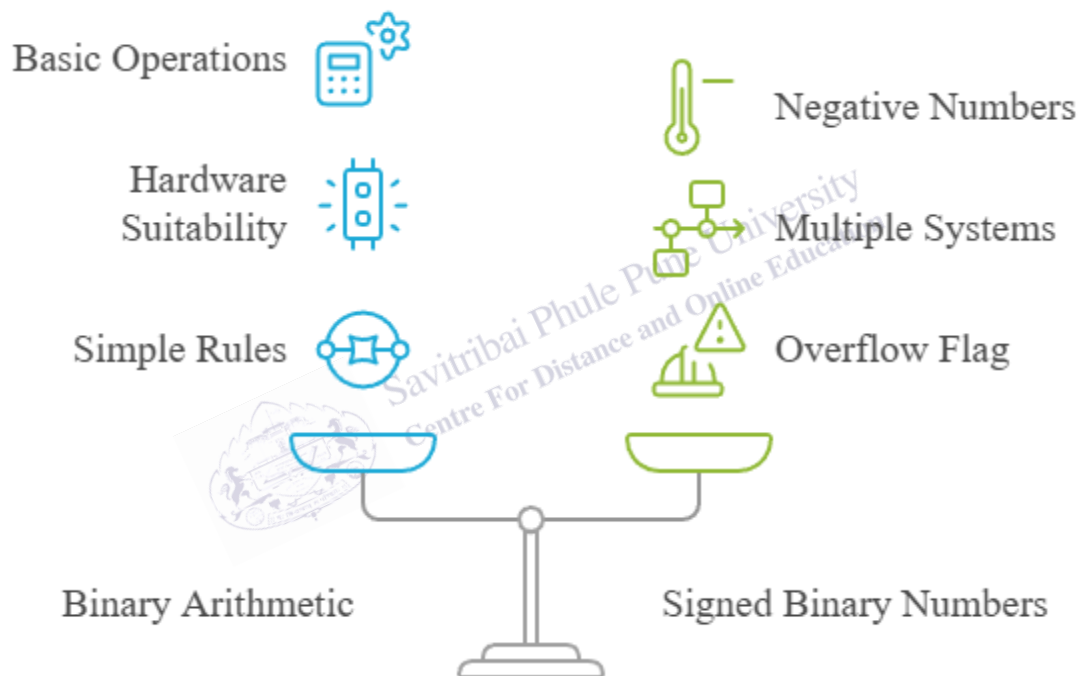
For example, take 0011, which is plus three in four bit form. Flip all bits to get 1100. Add one to get 1101. In four bit two's complement, 1101 stands for minus three. If you now add 0011 and 1101, the sum is 1 0000. The leftmost one is a carry out of the four bit range, so we drop it. The remaining four bits are 0000, which is zero. This confirms that plus three and minus three cancel each other.

2.4 Overflow And Range

In a fixed width system, such as four bit or eight bit two's complement, not every integer is representable. Four bits can represent only sixteen distinct patterns. In two's complement, these patterns cover a range from a certain negative minimum to a certain positive maximum. For four bits, the range is from minus eight to plus seven. For eight bits, the range is from minus one hundred and twenty eight to plus one hundred and twenty seven. The exact numbers come from powers of two.

Fig. 1

Binary Arithmetic and Signed Numbers



The diagram explains binary arithmetic and signed numbers, showing basic operations, simple rules, and hardware suitability. It highlights negative number representation, multiple number systems, and overflow handling, explaining how computers perform calculations and manage errors using signed binary arithmetic.

Overflow happens when a result falls outside this allowed range. For example, in four bit two's complement, the largest positive value is 0111, which is plus seven. If you add 0111 and 0001, you get 1000, which in this system stands for minus eight. The mathematical sum is plus eight,

but the four bit result is minus eight, which is clearly wrong. Hardware usually sets an overflow flag in such cases so that software can react if needed.



3.0 NATURE OR TYPE

3.1 Types Of Binary Operations

In the context of this unit, the main binary operations are addition and subtraction. Addition is used directly in the hardware, while subtraction is often performed by adding a negative value. Both operations may generate a carry or a borrow, depending on the bits involved. As you work more problems, you will notice that many patterns repeat, so that you can predict carries without full calculation.

Beyond addition and subtraction, binary numbers are also used in logical operations and bitwise operations. These include AND, OR, NOT, and XOR. Although they are not arithmetic in the narrow sense, they are related because the same bits are involved. For example, XOR is central to how an adder circuit works. As an online MCA student, you will later see how binary arithmetic operations and logical operations are combined inside an arithmetic logic unit.

3.2 Types Of Signed Representations

There are three classic ways to represent signed integers in binary. The first is sign magnitude. In sign magnitude, the leftmost bit is the sign bit, and the remaining bits hold the magnitude. This system is easy to understand, but it has two zeros: positive zero and negative zero. This double zero can confuse comparisons and arithmetic.

The second system is one's complement. Here, a negative number is formed by flipping every bit of the positive number. One's complement still has two zeros, because if you flip all bits of all zeros, you again get all ones, which is treated as negative zero. The third and most important system is two's complement. It uses the flip and add one rule. Two's complement has only one zero, and addition and subtraction work smoothly with the same hardware. That is why it is used in almost all modern processors.

3.3 Fixed Width And Sign Bit

Signed binary numbers are stored in fixed width registers, such as eight bit, sixteen bit, or thirty two bit registers. In a two's complement scheme, the leftmost bit is sometimes called the sign bit, because it helps determine whether the number is negative or non negative. If the leftmost bit is zero, the number is zero or positive. If the leftmost bit is one, the number is negative. This pattern holds because of how the range is divided.

When you expand a two's complement number to a larger width, you must preserve the sign. This process is called sign extension. To sign extend a shorter number, you copy its leftmost bit into the new higher bits. For example, if you take the four bit value 1101, which is minus three, and extend it to eight bits, you fill the new upper bits with ones, giving 11111101. This preserves

the value, because the extended pattern still decodes to minus three in eight bit two's complement form.

Fig. 2

How to represent signed integers in binary?



The diagram shows three ways to represent signed integers in binary: sign magnitude, one's complement, and two's complement. It explains how each method handles negative numbers, zeros, and hardware simplicity, highlighting why two's complement is most widely used.

3.4 Nature Of Errors In Manual Work

When students practice binary arithmetic by hand, most errors fall into a few simple types. One common error is to forget to carry a one during addition of bits. Another is to borrow from the wrong place during subtraction. A third is to take the complement of only some bits instead of all bits. These errors often arise from rushing or from not writing intermediate steps clearly.

To reduce such errors, it helps to work slowly at first and to check each column. You can also reverse the operation to verify your result. For example, if you subtract B from A and get C, you can add B and C to see whether the sum returns you to A. In the case of complements, you can take the two's complement twice and see whether you return to the original pattern. These small habits build a strong base for later courses where speed becomes more important.

4.0 EXAMPLES AND CASE STUDIES

4.1 Worked Example Of Binary Addition

Let us add two four bit numbers: 0101 and 0011, which are plus five and plus three. We write them one above the other and add from right to left. In the rightmost column, one plus one is zero with carry one. In the next column, zero plus one plus the carry one is again zero with carry one. In the third column, one plus zero plus carry one is zero with carry one. In the final column, zero plus zero plus carry one is one with no further carry. The result is 1000, which is eight in decimal.

This example shows how carries move through the columns. Even though we began with two small numbers, the result used a higher place value. If we had stayed with only three bits, the carry out of the top bit would have been lost, and we would have seen an incorrect result. This is a small reminder that the width of the register matters. The same idea appears when the processor uses fixed widths such as eight or thirty two bits.

4.2 Worked Example Of Binary Subtraction With Two's Complement

Now consider the subtraction 5 minus 3 in four bit two's complement. We first write the positive numbers: 0101 for five and 0011 for three. To subtract three, we find its two's complement. Flip all bits of 0011 to get 1100. Add one to get 1101, which is minus three. Now add 0101 and 1101. The sum is 1 0010. We drop the extra leftmost carry, leaving 0010, which is plus two. This matches the expected result.

If we instead try 3 minus 5, we start with 0011 for three and 0101 for five. We find the two's complement of 0101. Flip to get 1010. Add one to get 1011, which is minus five. Now add 0011 and 1011. The result is 1110. In four bit two's complement, 1110 represents minus two. We see that three minus five equals minus two, as expected. These examples show how a single adder can handle both addition and subtraction when we use two's complement representation.

4.3 Case Study: Temperature Sensor Readings

Imagine a small embedded system that reads temperatures from a sensor in a cold region. The sensor outputs an integer temperature in degrees Celsius, and the range is from minus twenty to plus fifteen. The microcontroller stores each reading in eight bit two's complement. When the sensor sends minus five degrees, the system must convert this to the correct bit pattern and store it.

Using eight bit two's complement, minus five is obtained by taking plus five, which is 00000101, flipping all bits to get 11111010, and adding one to get 11111011. When the microcontroller reads 11111011 from memory, it understands that this pattern stands for minus five. If a

programmer does not understand two's complement, they might misread 11111011 as a large positive number, which would cause errors in logging and control. This case shows why binary arithmetic knowledge is important in real devices.

4.4 Case Study: Bank Account Balances

Consider an online banking system that stores account balances as signed thirty two bit integers. A positive balance means the customer has money in the account. A negative balance may mean that the customer has used an overdraft facility. When a transaction occurs, the system adds or subtracts an amount from the stored balance using two's complement arithmetic.

If a bug causes the program to add an amount that is too large, the result may overflow the allowed range. A very large positive balance could wrap around into a negative number. Good systems include checks for overflow and do not trust the raw result if an overflow flag is set. Designers also choose data types large enough that real balances cannot reach the limits in normal use. This case study links the abstract idea of overflow to a situation that matters to users and banks.

4.5 Common Mistakes In Student Work

When students work through examples like those above, certain mistakes appear often. One mistake is mixing up the order of steps in the two's complement process, such as adding one before flipping bits, or forgetting one of the steps entirely. Another is using the wrong number of bits, which changes the sign and the value. A third mistake arises when students forget to drop the extra carry after adding a number and its negative.

To avoid these mistakes, it is helpful to adopt a fixed method and repeat it every time. Write the original binary number. Flip every bit carefully. Add one in a clear column addition. Check that the leftmost bit is one for a negative number in two's complement. When adding a number and its two's complement, expect a result of all zeros after discarding the carry. If you see a different pattern, you know that somewhere a step was skipped or a digit was copied wrongly.

4.6 Practice Strategy For Students

A good practice strategy can turn a difficult topic into a familiar friend. Start by working a small set of very easy problems, such as adding or subtracting numbers that you already know in decimal. Write each step, including carries and borrows, so that your eye becomes used to the flow of bits across the page. When you make a mistake, circle it and write a short note about why it happened. This reflection slowly trains your mind to avoid the same trap next time.

Next, mix your problems so that they include both positive and negative numbers. For example, try sums like plus three plus minus two, or minus four plus minus three, using two's complement. By doing this, you teach yourself that two's complement is not a strange special

case but a normal part of arithmetic. Finally, practise under light time pressure, as in a quiz environment. Even five minutes of timed work a day can improve both your speed and your confidence before assessments.

4.7 Summary Of The Unit

This unit has taken you from the idea of binary numbers to the full method of two's complement arithmetic. You have seen how hardware friendly rules for addition and subtraction can be applied on paper. You have also seen that signed representations like sign magnitude and one's complement exist, but that two's complement is superior for most real systems. Step by step, the unit has linked theory to examples such as sensor readings and bank balances.

When you move to later subjects such as digital logic, microprocessors, or systems programming, these ideas will return in new forms. Instruction sets use two's complement for calculating addresses. Compilers assume fixed width ranges when they optimise loops. Debugging tools display registers as binary or hexadecimal fields that you can interpret only if you understand how bits map to values. By mastering the simple rules in this unit now, you prepare yourself for many advanced topics across the MCA curriculum.

As you study other areas of computing, try to notice whenever numbers are being stored or moved. When you see a hexadecimal constant, imagine the underlying binary pattern. When a language manual speaks about the size of an integer type, think about how many distinct values that size can represent using two's complement. This habit of linking higher level concepts back to bit level details will make your understanding deeper and more robust over time.

Regular revision, self testing, and discussion with classmates make the abstract ideas feel more natural and less frightening during exams and interviews. You can also teach a small part of the topic to a friend; explaining a method aloud forces you to organise your thoughts and expose any remaining gaps clearly.

1. Binary arithmetic works with base two and uses simple carry and borrow rules.
2. Signed binary numbers let us represent both positive and negative integers inside a fixed number of bits.
3. Two's complement is the standard method for storing signed integers in modern processors.
4. The range of an n bit two's complement system is from minus two to the power of n minus one up to plus two to the power of n minus one minus one.
5. Overflow happens when the true mathematical result is outside the representable range for the chosen bit width.
6. Subtraction can be carried out by adding the two's complement of the number to be subtracted.

7. Sign extension keeps the value of a two's complement number when it is moved to a larger register.
8. Careful step by step work and reverse checking reduce common student errors in binary calculations.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple meaning
Binary arithmetic	Doing calculations on numbers written with only the digits zero and one
Signed number	A number that includes information about whether it is positive or negative
Two's complement	A bit pattern method for representing negative integers using flip and add one
One's complement	A method that forms negatives by flipping every bit of the positive pattern
Sign magnitude	Representation where the leftmost bit holds the sign and other bits hold size
Sign bit	The leftmost bit that shows whether a two's complement number is negative
Sign extension	Copying the sign bit into new higher bits when widening a two's complement value
Overflow	Error that happens when a result is outside the allowed range of values
Fixed width	Using a set number of bits, such as eight or thirty two, to store each number
Carry	Extra one that moves to the next column in addition of two binary digits